Kiến trúc đa thuê bao và thiết kế ứng dụng Kanban ở Cấp độ 3

Tóm tắt

Đề tài hướng tới thiết kế, xây dựng và demo một ứng dụng quản lý công việc Kanban đa thuê bao đạt Cấp độ 3 trong mô hình trưởng thành SaaS của Chong và Carraro. Các tổ chức sử dụng chung mã nguồn, một instance ứng dụng và cơ sở dữ liệu; dữ liệu được phân tách logic bằng Tenant ID, còn khác biệt về giao diện và nghiệp vụ được xử lý lúc chạy thông qua metadata. Báo cáo dùng mô hình bốn cấp độ làm khung phân tích, từ đó xác định kiến trúc mục tiêu, cơ chế cô lập và tiêu chí kiểm chứng demo. Cấp độ 4 được đặt ở hướng phát triển với cân bằng tải, mở rộng ngang và phân bố dữ liệu theo nhu cầu.

Phạm vi báo cáo

Hai chương trình bày cơ sở lý thuyết và kiến trúc phục vụ mục tiêu Cấp độ 3. Chương 1 làm rõ khái niệm, mô hình trưởng thành và cơ sở kinh tế. Chương 2 chuyển các tiêu chí đó thành thiết kế và kế hoạch kiểm chứng cho ứng dụng Kanban. Việc xác nhận ứng dụng đạt Cấp độ 3 cần dựa trên mã nguồn, cấu hình triển khai và kết quả chạy thử tương ứng với từng tiêu chí.

Từ khóa: SaaS; multi-tenant; tenant; runtime metadata; Tenant ID; mô hình trưởng thành.

Mục lục

CHƯƠNG 1 CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH TRƯỞNG THÀNH 3
1.1 SaaS và tiến trình chia sẻ tài nguyên 3
1.2 Các khái niệm nền tảng 5
1.3 Phân biệt multi user và multi tenant 7
1.4 Bốn cấp độ trưởng thành kiến trúc SaaS 8
1.5 Mô hình khái niệm ba tầng 11
1.6 Tổng chi phí sở hữu từ góc nhìn nhà cung cấp 12
1.7 Tổng kết chương 14
CHƯƠNG 2 KIẾN TRÚC MỤC TIÊU VÀ KIỂM CHỨNG CẤP ĐỘ 3 15
2.1 Mục tiêu và phạm vi demo 15
2.2 Lựa chọn cách chia sẻ ứng dụng và dữ liệu 16
2.3 Cấu trúc ứng dụng theo mô hình ba tầng 17
2.4 Xác định và cô lập ngữ cảnh tenant 18
2.5 Thiết kế database dùng chung 19
2.6 Tùy biến runtime qua metadata 21
2.7 Tài nguyên dùng chung và giới hạn của kiến trúc 22
2.8 Kế hoạch kiểm chứng demo theo tiêu chí Cấp độ 3 23
2.9 Hướng phát triển lên Cấp độ 4 26
2.10 Tổng kết chương 26
Tài liệu tham khảo 27

CHƯƠNG 1 CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH TRƯỞNG THÀNH

Chương này xác lập những khái niệm cần thiết để trả lời ba câu hỏi: hệ thống phục vụ ai, tài nguyên nào được dùng chung và điều kiện nào cho phép định vị demo ở Cấp độ 3. Mô hình bốn cấp độ được sử dụng xuyên suốt để liên kết cơ sở lý thuyết với lựa chọn kiến trúc và tiêu chí đánh giá.

1.1 SaaS và tiến trình chia sẻ tài nguyên

1.1.1 Phần mềm dưới dạng dịch vụ

Phần mềm dưới dạng dịch vụ, hay Software as a Service (SaaS), là mô hình cung cấp phần mềm trong đó nhà cung cấp chịu trách nhiệm triển khai, vận hành và duy trì ứng dụng, còn khách hàng truy cập các chức năng thông qua mạng. Theo định nghĩa của Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST), SaaS cho phép người sử dụng khai thác các ứng dụng của nhà cung cấp đang hoạt động trên hạ tầng đám mây thông qua giao diện khách hoặc giao diện lập trình. Người sử dụng không trực tiếp quản lý hạ tầng nền như mạng, máy chủ, hệ điều hành và lưu trữ, nhưng có thể được phép điều chỉnh một số cấu hình của ứng dụng.

Xét về phương thức chuyển giao, SaaS thay đổi cách khách hàng tiếp cận và sử dụng phần mềm. Với hình thức cài đặt tại chỗ, tổ chức thường phải chuẩn bị môi trường triển khai, thực hiện cài đặt và duy trì hoạt động của hệ thống. Với SaaS, các trách nhiệm này được chuyển phần lớn sang nhà cung cấp. Khách hàng tập trung vào việc sử dụng chức năng nghiệp vụ, quản lý người dùng và thiết lập những cấu hình nằm trong phạm vi dịch vụ cho phép.

Nhà cung cấp duy trì việc phát hành, cập nhật và hỗ trợ ứng dụng; tổ chức khách hàng quản lý người dùng và các cấu hình được dịch vụ cho phép. Vì vậy, cùng một sản phẩm SaaS cần đáp ứng nhu cầu riêng của nhiều tổ chức mà vẫn kiểm soát được chi phí vận hành. Thu phí định kỳ là hình thức kinh doanh phổ biến, nhưng riêng phương thức thanh toán không quyết định cách tổ chức kiến trúc.

SaaS mô tả cách cung cấp phần mềm; multi-tenancy mô tả cách nhiều thuê bao sử dụng chung tài nguyên. Một dịch vụ SaaS vẫn có thể bố trí ứng dụng riêng cho từng khách hàng. Ngược lại, gọi một hệ thống là đa thuê bao cần chỉ rõ tài

nguyên dùng chung và cơ chế giữ riêng dữ liệu, cấu hình, quyền truy cập của từng tổ chức.

1.1.2 Từ chia sẻ máy tính đến chia sẻ ứng dụng

Kiến trúc đa thuê bao kế thừa nhiều ý tưởng từ các mô hình chia sẻ tài nguyên điện toán trước đó. Theo phần tổng quan của Kabbedijk và cộng sự (2015), những tiền đề đáng chú ý gồm chia sẻ thời gian trên máy tính lớn, cung cấp ứng dụng từ xa theo mô hình ASP và các ứng dụng web đa người dùng. Các mô hình này thể hiện những cách khác nhau để phục vụ nhiều đối tượng trên một tập tài nguyên, đồng thời đặt ra những yêu cầu khác nhau về cấu hình và quản lý.

Chia sẻ thời gian trên máy tính lớn. Từ thập niên 1960, chi phí đầu tư và khai thác máy tính lớn tạo ra nhu cầu sử dụng chung năng lực tính toán. Mô hình chia sẻ thời gian (time-sharing) cho phép nhiều người sử dụng tiếp cận tài nguyên xử lý của cùng một hệ thống thông qua cơ chế phân phối thời gian thực thi. Ý tưởng nền tảng là nâng cao mức sử dụng tài nguyên và phân bổ chi phí cho nhiều đối tượng. Tuy nhiên, ranh giới sử dụng ở giai đoạn này chủ yếu được tổ chức theo tài khoản và môi trường tính toán, chưa tương ứng đầy đủ với khái niệm thuê bao ở tầng ứng dụng SaaS.

Nhà cung cấp dịch vụ ứng dụng. Khoảng thập niên 1990, mô hình Application Service Provider (ASP) phát triển theo hướng nhà cung cấp lưu trữ và vận hành ứng dụng thay cho khách hàng. Người sử dụng tiếp cận phần mềm từ xa, trong khi nhà cung cấp quản lý môi trường thực thi. Với những ứng dụng chưa được thiết kế để phục vụ nhiều tổ chức trong cùng một bản thể thực thi, mỗi khách hàng thường cần một máy hoặc một tiến trình riêng. Cách tổ chức này hỗ trợ cung cấp phần mềm tập trung nhưng làm tăng số môi trường phải cấu hình, cập nhật và theo dõi khi số khách hàng mở rộng.

Ở cách cài đặt tại chỗ (on-premise), mỗi tổ chức tự quản lý môi trường của mình. Mô hình ASP chuyển trách nhiệm vận hành ứng dụng sang nhà cung cấp, nhưng các bản ứng dụng dành riêng vẫn có thể tồn tại. Các ứng dụng web sau đó cho thấy khả năng phục vụ nhiều người dùng trên cùng nền tảng; SaaS đa thuê bao bổ sung việc quản trị dữ liệu và cấu hình theo từng tổ chức. Đây là những hướng phát triển có liên hệ và có thể cùng tồn tại, không phải một chuỗi trong đó mô hình sau thay thế hoàn toàn mô hình trước.

Ứng dụng web đa người dùng. Sự phát triển của ứng dụng web cho thấy khả năng phục vụ nhiều người dùng trên cùng một nền tảng phần mềm. Các tài khoản có thể cùng sử dụng một tập chức năng và hạ tầng xử lý, trong khi dữ liệu cá nhân được quản lý theo quyền truy cập. Mô hình này mở rộng khả năng chia sẻ tài nguyên ở tầng ứng dụng, nhưng việc phục vụ nhiều tài khoản chưa tự nó giải quyết yêu cầu của nhiều tổ chức khách hàng. Những nhu cầu như quản trị thành viên, cấu hình quy trình hoặc phân tách dữ liệu theo tổ chức đòi hỏi một phạm vi quản lý cao hơn tài khoản cá nhân.

SaaS đa thuê bao. Kiến trúc đa thuê bao phát triển theo hướng kết hợp khả năng dùng chung tài nguyên với việc quản lý nhu cầu riêng của các khách hàng. Mỗi thuê bao được xác định như một đơn vị sử dụng có ngữ cảnh dữ liệu, cấu hình và chính sách truy cập riêng. Hệ thống có thể sử dụng nền mã và một phần tài nguyên chung, đồng thời điều chỉnh hành vi theo ngữ cảnh thuê bao.

Trong phạm vi đề tài, ý nghĩa của tiến trình này là chuyển đơn vị quản lý từ tài khoản cá nhân hoặc bản cài đặt riêng sang tổ chức thuê dịch vụ trên một ứng dụng chung. Mô hình trưởng thành ở Mục 1.4 mô tả cụ thể sự thay đổi đó qua mã nguồn, cách cấu hình, instance ứng dụng và dữ liệu.

1.2 Các khái niệm nền tảng

1.2.1 Tenant và user

Tenant (thuê bao) trong đề tài là một tổ chức hoặc doanh nghiệp đăng ký sử dụng dịch vụ và có một định danh riêng. Tổ chức có phạm vi dữ liệu, danh sách thành viên, quyền quản trị và cấu hình riêng về giao diện hoặc nghiệp vụ trong giới hạn nền tảng hỗ trợ. Với ứng dụng Kanban, một workspace đại diện cho một tenant; các bảng và thẻ công việc của workspace thuộc phạm vi tenant đó.

User (người dùng) là cá nhân truy cập hệ thống. Quyền của user được xác định trong quan hệ thành viên với tenant: cùng một cá nhân có thể là quản trị viên ở tổ chức A nhưng chỉ là thành viên ở tổ chức B. Do đó, user_id trả lời câu hỏi “ai đang thao tác”, còn tenant_id trả lời câu hỏi “thao tác thuộc tổ chức nào”. Hai định danh không được dùng thay thế cho nhau.

Ví dụ, công ty A có 20 nhân viên và công ty B có 15 nhân viên. Hệ thống phục vụ hai tenant và 35 tài khoản người dùng nếu các tài khoản không trùng nhau. Việc thêm nhân viên cho công ty A làm tăng số user của A; việc thêm công ty C

tạo một tenant mới với cấu hình và dữ liệu độc lập. Quy mô trên đồ thị TCO của báo cáo được tính bằng số tenant, không phải số tài khoản.

Tenant ID là khóa định danh tổ chức trong hệ thống. Trong mô hình bảng dùng chung, các bản ghi thuộc tenant mang khóa này để phục vụ kiểm soát truy cập. Tuy nhiên, chỉ thêm cột tenant_id chưa tạo ra cô lập: ứng dụng phải xác thực quan hệ thành viên và thực thi giới hạn đọc, ghi theo tenant hiện hành.

1.2.2 Service và instance

Dịch vụ (Service): Tập hợp các năng lực và chức năng nghiệp vụ phần mềm được nhà cung cấp đóng gói, vận hành và cung ứng cho thuê bao thông qua mạng Internet. Khách hàng tương tác và đánh giá hệ thống dựa trên chất lượng dịch vụ và tính năng nghiệp vụ mà nhà cung cấp mang lại, hoàn toàn độc lập với chi tiết kỹ thuật về hạ tầng bên dưới.

Instance là một bản thành phần phần mềm đang chạy và được cấp tài nguyên để xử lý yêu cầu. Ví dụ, một tiến trình Web Server hoặc Application Server chạy backend Kanban là một instance ứng dụng; một tiến trình Database Server như PostgreSQL là một instance hệ quản trị cơ sở dữ liệu. Cần gọi rõ loại instance đang xét thay vì chỉ dùng cụm "thực thể vận hành".

Một Web Server có thể xử lý yêu cầu của cả tenant A và B. Hai tiến trình backend chạy cùng một bộ mã nguồn vẫn là hai instance ứng dụng khác nhau. Tương tự, một Database Server có thể quản lý nhiều database; vì vậy "dùng chung Database Server" chưa đủ để kết luận các tenant dùng chung một database hoặc cùng các bảng dữ liệu.

Trong kiến trúc mục tiêu của demo, "một instance" được hiểu là một instance backend thực thi nghiệp vụ Kanban cho đồng thời nhiều tenant. Trình duyệt của từng user không phải là instance backend riêng của tenant. Bằng chứng triển khai cần chỉ rõ tiến trình hoặc container backend và nguồn dữ liệu mà các yêu cầu thực sự sử dụng.

1.2.3 Định nghĩa đa thuê bao trong phạm vi đề tài

Kabbedijk và cộng sự (2015) xem đa thuê bao là thuộc tính của hệ thống trong đó nhiều khách hàng chia sẻ trong suốt tài nguyên như dịch vụ, ứng dụng, dữ liệu hoặc phần cứng nhằm giảm chi phí, đồng thời có thể cấu hình theo nhu cầu riêng.

Định nghĩa này giúp xác định cả phần dùng chung và phạm vi riêng của mỗi thuê bao, thay vì đồng nhất đa thuê bao với việc có nhiều tài khoản.

- Thứ nhất, tính trong suốt - “transparently”. Việc chia sẻ tài nguyên được che giấu khỏi cách người dùng thực hiện nghiệp vụ thông thường. Người dùng tương tác với không gian làm việc và các chức năng được cấp quyền mà không phải trực tiếp quản lý cách tài nguyên được phân bổ giữa các khách hàng. Hệ thống cần duy trì ngữ cảnh sử dụng riêng để sự hiện diện của các thuê bao khác không làm thay đổi ý nghĩa của dữ liệu và thao tác trong phạm vi đang xét.

Tính trong suốt không tự tạo ra sự độc lập về hiệu năng. Khi tài nguyên hữu hạn được chia sẻ, tải của một thuê bao vẫn có thể ảnh hưởng đến thuê bao khác. Vì vậy, trải nghiệm ổn định tương ứng với mục tiêu chất lượng dịch vụ phải được hỗ trợ bằng cơ chế quản lý tài nguyên và kiểm chứng thực nghiệm. Cần phân biệt thuộc tính che giấu sự chia sẻ với mức độ cô lập hiệu năng thực tế.

Chia sẻ tài nguyên cũng không đồng nghĩa với chia sẻ quyền truy cập dữ liệu. Hai thuê bao có thể sử dụng cùng một cơ sở dữ liệu trong khi mỗi bên chỉ được thao tác trên dữ liệu thuộc phạm vi được cấp quyền. Sự phân biệt này là cơ sở cho yêu cầu cô lập dữ liệu trong thiết kế đa thuê bao.

Đề tài cụ thể hóa định nghĩa trên bằng mục tiêu Cấp độ 3: nhiều tổ chức dùng chung ứng dụng và database, khác biệt được biểu diễn bằng metadata theo tenant, dữ liệu và quyền truy cập được kiểm soát qua ngữ cảnh thuê bao. Khả năng tùy biến có giới hạn rõ ràng; không yêu cầu tenant tự sửa mã nguồn hoặc thực thi mã tùy ý trên hệ thống.

1.3 Phân biệt multi user và multi tenant

Multi-user là khả năng cho nhiều cá nhân sử dụng phần mềm với danh tính, phiên làm việc và quyền hạn tương ứng. Phần mềm nội bộ của một công ty có hàng trăm tài khoản vẫn có thể chỉ phục vụ một tenant. Multi-tenant bổ sung nhiều tổ chức khách hàng, mỗi tổ chức có phạm vi dữ liệu, quản trị thành viên và cấu hình riêng. Một ứng dụng SaaS có thể đồng thời là multi-user và multi-tenant.

Bảng 1.1 Đối chiếu các khái niệm dễ nhầm lẫn

Khái niệm	Đơn vị đang xét	Ý nghĩa đối với đề tài
Single-tenant	Một tổ chức tại tầng đang xét	Ứng dụng hoặc dữ liệu được dành riêng cho một tenant.
Multi-user	Nhiều cá nhân	Nhiều tài khoản cùng sử dụng; chưa đủ chứng minh có nhiều tổ chức độc lập.
Multi-instance	Nhiều bản ứng dụng đang chạy	Mô tả số bản chạy; mỗi bản có thể phục vụ một hoặc nhiều tenant.
Multi-tenant	Nhiều tổ chức khách hàng	Quản lý dữ liệu, thành viên và cấu hình theo từng tenant trên tài nguyên dùng chung.

Multi-instance không phải một cấp độ trưởng thành riêng. Ở Cấp độ 2, mỗi tenant có instance của mình; ở Cấp độ 4, một nhóm instance có thể cùng phục vụ nhiều tenant. Vì vậy, số bản chạy cần được xem cùng quan hệ tenant–instance và cơ chế tổ chức dữ liệu.

Ví dụ đối chiếu: hệ thống Kanban nội bộ chỉ có công ty A, dù nhiều nhân viên cùng đăng nhập, chứng minh multi-user. Khi công ty B được khởi tạo, B tự quản lý thành viên và giao diện, dữ liệu B không xuất hiện trong phiên A, trong khi cả hai dùng chung backend và database, hệ thống mới thể hiện các thuộc tính đa thuê bao theo mục tiêu của đề tài.

1.4 Bốn cấp độ trưởng thành kiến trúc SaaS

Chong và Carraro (2006) đề xuất bốn cấp độ trưởng thành dựa trên khả năng cấu hình, hiệu quả phục vụ đa thuê bao và khả năng mở rộng. Báo cáo dùng mô hình này làm trục phân tích; phạm vi dùng chung database và cách biểu diễn metadata được đặc tả rõ cho demo để có thể kiểm chứng. Đây là khung định vị kiến trúc, không phải một chứng nhận độc lập về bảo mật hoặc hiệu năng.

1.4.1 Cấp độ 1 mã tùy biến và môi trường riêng

Mỗi khách hàng sử dụng một bản ứng dụng được tùy biến riêng, thường đi kèm nhánh mã nguồn, instance ứng dụng và database riêng. Việc sửa nghiệp vụ của A có thể tạo ra phiên bản khác với B. Nhà cung cấp phải quản lý các biến thể, kiểm thử và nâng cấp trên những môi trường tương ứng.

Ưu điểm là dễ đáp ứng yêu cầu rất riêng ở quy mô nhỏ; hạn chế là chi phí duy trì tăng khi các phiên bản khác biệt tích lũy. Khả năng có nhiều khách hàng sử dụng các bản cài đặt này chưa chứng minh nhiều tenant cùng được phục vụ bởi một instance. Đề tài không chọn cách duy trì mã nguồn riêng theo tổ chức.

1.4.2 Cấp độ 2 mã nguồn chung và cấu hình trước khi chạy

Cấp độ 2 chuẩn hóa một codebase dùng chung. Theo phạm vi đối chiếu của báo cáo, khác biệt của từng khách hàng được đặt trong file cấu hình trước khi chạy, chẳng hạn thông tin nhận diện giao diện, các chức năng bật tắt và thông số kết nối. Mỗi tenant vẫn có instance ứng dụng và database riêng; thay vì sửa mã nguồn cho A và B, nhà cung cấp triển khai cùng mã với các file cấu hình khác nhau.

Ví dụ, App A đọc file cấu hình A và kết nối DB A, còn App B đọc file cấu hình B và kết nối DB B. Cùng mã nguồn giúp giảm phân kỳ phiên bản nhưng không loại bỏ việc vận hành nhiều bản ứng dụng. Thay đổi cấu hình có thể đòi hỏi khởi động lại hoặc triển khai lại tùy cơ chế hiện thực; điểm phân biệt ở đây là cấu hình gắn với từng bản chạy riêng trước khi phục vụ.

1.4.3 Cấp độ 3 ứng dụng chung và metadata lúc chạy

Cấp độ 3 là mục tiêu của đề tài. Một instance backend phục vụ nhiều tenant trên cùng mã nguồn. Mỗi yêu cầu mang một ngữ cảnh tenant đã được xác thực; ứng dụng dựa vào ngữ cảnh đó để áp dụng metadata và kiểm soát dữ liệu. Trong phương án được chọn, các tenant cùng dùng một database và các bảng nghiệp vụ có tenant_id để phân tách logic.

Metadata mô tả những điểm biến đổi được nền tảng cho phép, chẳng hạn tên workspace, màu giao diện, chức năng khả dụng hoặc giới hạn công việc đang làm. Ứng dụng đọc và diễn giải metadata theo tenant tại thời điểm xử lý. Cập nhật cấu hình A làm thay đổi hành vi được hỗ trợ cho A trong khi B giữ cấu hình của mình, không cần tạo nhánh mã hoặc instance riêng.

Ví dụ, A bật giới hạn số thẻ đang thực hiện, B không bật; cùng một mã xử lý Kanban đọc cấu hình của tenant hiện hành và áp dụng quy tắc tương ứng. Sự khác biệt này phải được thực thi ở backend. Chỉ đổi màu trên trình duyệt hoặc ẩn một nút chức năng chưa chứng minh quyền sử dụng tính năng đã được kiểm soát.

Ba thuộc tính cần kiểm chứng đồng thời là: mã nguồn chung với tùy biến runtime qua metadata; một instance phục vụ đồng thời nhiều tenant; database dùng chung có dữ liệu gắn Tenant ID và cơ chế cô lập ngữ cảnh. Một hình vẽ hoặc cột tenant_id đơn lẻ không đủ để kết luận đã đạt Cấp độ 3.

1.4.4 Cấp độ 4 mở rộng ngang

Cấp độ 4 kế thừa khả năng cấu hình và nhận biết tenant của Cấp độ 3, đồng thời tổ chức một tập hợp instance ứng dụng để mở rộng năng lực phục vụ. Yêu cầu được phân phối qua bộ cân bằng tải; việc bổ sung instance không làm thay đổi cách nhận diện tenant hoặc quyền truy cập. Ứng dụng cần quản lý trạng thái và dữ liệu sao cho một tenant có thể được xử lý đúng khi yêu cầu đi qua các instance khác nhau.

Hướng phát triển của đề tài gồm nhiều instance backend hoặc container/pod sau bộ cân bằng tải, cùng phương án mở rộng dữ liệu như phân vùng, bản sao đọc hoặc cơ sở dữ liệu phân tán khi nhu cầu thực tế đòi hỏi. Pod và database phân tán là lựa chọn hiện thực, không phải tên gọi bắt buộc của mô hình gốc. Riêng việc đóng gói ứng dụng trong container không chứng minh đã đạt Cấp độ 4.

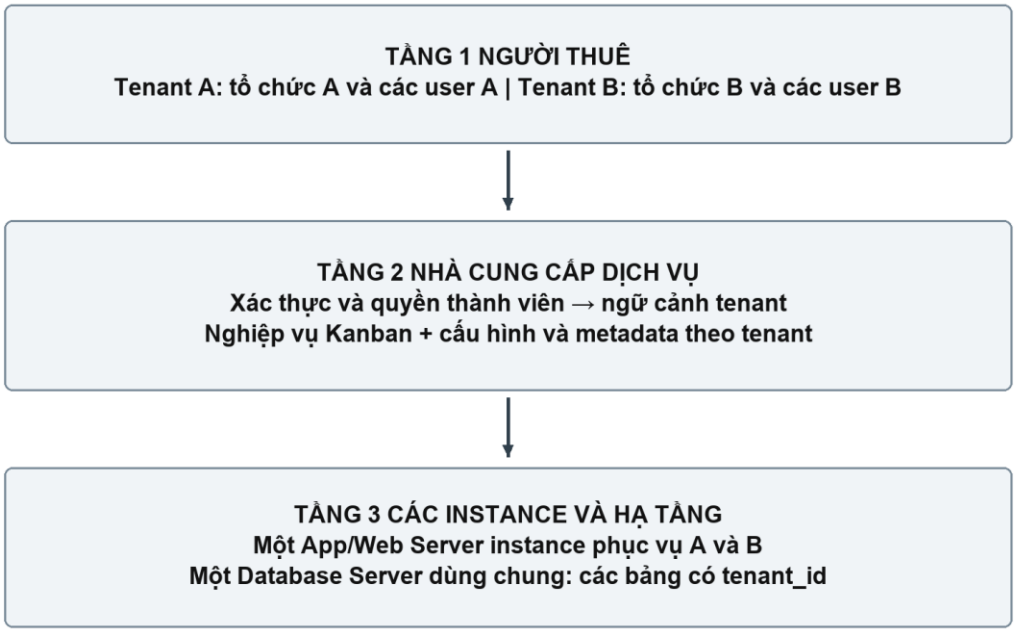
Bảng 1.2 So sánh bốn cấp độ trong phạm vi báo cáo

Cấp độ	Mã và cấu hình	Ứng dụng và dữ liệu	Vai trò
1	Mã tùy biến theo khách hàng	App riêng; DB riêng	Bối cảnh xuất phát
2	Codebase chung; file cấu hình trước khi chạy	App riêng; DB riêng theo tenant	Mốc đối chiếu
3	Codebase chung; metadata theo tenant lúc chạy	Một App chung; DB và bảng chung có tenant_id	Mục tiêu demo
4	Kế thừa cấu hình động của Cấp độ 3	Nhiều App qua cân bằng tải; mở rộng dữ liệu theo tải	Hướng phát triển

Sự khác nhau quyết định giữa Cấp độ 2 và 3 nằm ở quan hệ tenant–instance và thời điểm áp dụng biến thể. Cấp độ 2 gắn cấu hình của từng tenant vào bản chạy riêng. Cấp độ 3 chọn metadata theo ngữ cảnh trong cùng bản chạy đang phục vụ nhiều tenant. Khả năng có một repository mã nguồn hoặc một file cấu hình chung không tự phân biệt được hai cấp độ này.

1.5 Mô hình khái niệm ba tầng

Mô hình ba tầng trong báo cáo thể hiện quan hệ giữa người thuê, nhà cung cấp và các thành phần đang chạy. Đây là cách nhìn theo vai trò cung ứng dịch vụ, giúp giải thích tenant sử dụng gì và tài nguyên nào được chia sẻ. Nó không thay thế cách chia giao diện, xử lý nghiệp vụ và truy cập dữ liệu khi thiết kế bên trong ứng dụng.



Hình 1.1 Mô hình ba tầng của kiến trúc đa thuê bao mục tiêu

Tầng người thuê gồm các tổ chức A, B và các user thuộc mỗi tổ chức. Quản trị viên tenant cấu hình những điểm biến đổi được nền tảng cho phép; user thao tác theo quyền thành viên. Tenant không trực tiếp quyết định việc cấp phát CPU hoặc sửa cấu hình database của nhà cung cấp.

Tầng nhà cung cấp dịch vụ gồm chức năng Kanban, quản lý tenant, thành viên và cấu hình. Tầng này xác thực yêu cầu, xác định tenant hiện hành, áp dụng metadata và kiểm soát quyền trước khi thực hiện nghiệp vụ. Cùng một dịch vụ có thể cung cấp hành vi phù hợp cho A và B bằng việc chọn dữ liệu và cấu hình theo ngữ cảnh.

Tầng các instance và hạ tầng thể hiện những thành phần thực thi, ví dụ Web/Application Server và Database Server. Ở Cấp độ 2, A và B được bố trí vào các instance ứng dụng và DB riêng. Ở Cấp độ 3, cả hai đi vào cùng instance backend và database; ranh giới được thực thi bằng định danh tenant, quyền và

chính sách dữ liệu. Khi hướng đến Cấp độ 4, tầng này mở rộng thành tập instance phối hợp qua cân bằng tải.

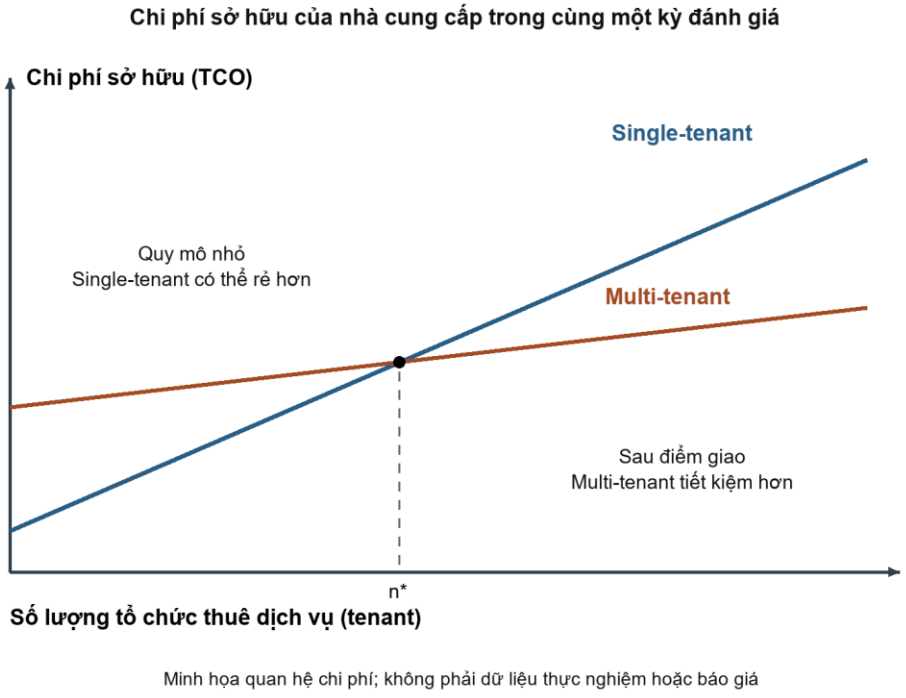
Mô hình ba tầng liên kết tenant phát sinh yêu cầu, dịch vụ xử lý, instance tiếp nhận và dữ liệu bị tác động. Quan hệ này được dùng để tổ chức bằng chứng kiểm chứng ở Chương 2.

1.6 Tổng chi phí sở hữu từ góc nhìn nhà cung cấp

1.6.1 Phạm vi và các trục của đồ thị

Tổng chi phí sở hữu (Total Cost of Ownership, TCO) ở đây là tổng chi phí mà nhà cung cấp dịch vụ phải bỏ ra để xây dựng, triển khai và vận hành hệ thống trong một kỳ đánh giá xác định. Các khoản cần xét gồm phát triển nền tảng, hạ tầng, cấp phát, bảo trì, giám sát, sao lưu và xử lý sự cố. Đây không phải tiền thuê bao mà khách hàng thanh toán hoặc TCO riêng của người đi thuê.

Trục hoành biểu diễn quy mô khách hàng, tính bằng số tổ chức thuê dịch vụ (số tenant). Trục tung biểu diễn tổng chi phí của nhà cung cấp trong cùng kỳ đánh giá và cùng đơn vị tiền tệ hoặc chi phí quy đổi. So sánh phải giữ nhất quán yêu cầu chức năng, mức dịch vụ và tải trung bình mỗi tenant; nếu chỉ một mô hình phải gánh tải cao hơn thì hai đường không còn phản ánh cùng bài toán.



Hình 1.2 Chi phí sở hữu của nhà cung cấp theo số tenant

Hình 1.2 là đồ thị khái niệm. Độ dốc và vị trí điểm giao minh họa cơ chế kinh tế, không biểu diễn một mức tiết kiệm hay số lượng tenant đã đo được. Đồ thị dùng tổng chi phí, do đó đường multi-tenant vẫn tăng khi quy mô tăng; không thể suy ra rằng thêm tenant mới có chi phí bằng không.

1.6.2 Chi phí khởi đầu và chi phí tăng thêm

Ở quy mô nhỏ, triển khai single-tenant có thể đơn giản hơn vì ứng dụng chưa cần đầy đủ cơ chế phân giải tenant, cô lập bảng dùng chung và quản trị metadata theo tổ chức. Trong giả thiết của đồ thị, chi phí khởi đầu của single-tenant vì vậy thấp hơn. Multi-tenant phải đầu tư sớm cho những cơ chế này, nên tổng chi phí ban đầu có thể cao hơn dù chỉ phục vụ ít khách hàng.

Khi quy mô tăng, single-tenant phải bổ sung và duy trì nhiều môi trường: instance, database, bản sao lưu và quy trình cập nhật tương ứng. Multi-tenant có thể tiếp nhận tenant mới bằng thêm định danh, thành viên, metadata và dữ liệu khởi tạo trên năng lực còn sẵn có. Nhiều tổ chức chia sẻ tài nguyên và công cụ vận hành, làm giảm chi phí tăng thêm bình quân trong điều kiện tải và chất lượng dịch vụ được kiểm soát.

Một vấn đề kinh tế khác là cấp phát tài nguyên theo nhu cầu đỉnh. Nếu mỗi khách hàng được chuẩn bị một lượng tài nguyên riêng đủ đáp ứng thời điểm tải cao, phần năng lực đó có thể chưa được khai thác trong những khoảng thời gian còn lại. Khi nhiều khách hàng sử dụng tài nguyên chung và có đỉnh tải không trùng nhau, hệ thống có cơ hội phân bổ năng lực linh hoạt hơn. Cơ chế này thường được giải thích thông qua nguyên lý ghép kênh thống kê (statistical multiplexing): nhu cầu tổng hợp có thể được phục vụ hiệu quả hơn so với việc dự phòng độc lập cho từng khách hàng.

Điểm giao n* là quy mô tại đó hai phương án có TCO bằng nhau theo các giả thiết đang xét. Trước n*, lợi thế triển khai ban đầu của single-tenant có thể lớn hơn phần tiết kiệm do chia sẻ. Sau n*, chi phí khởi đầu cao hơn của multi-tenant được bù bởi chi phí tăng thêm thấp hơn, nên tổng chi phí của nó thấp hơn single-tenant. n* là ngưỡng cân bằng chi phí giữa hai phương án, không phải điểm hòa vốn doanh thu của dịch vụ.

1.6.3 Giới hạn của kết luận kinh tế

Tuy nhiên, mức tiết kiệm phụ thuộc vào điều kiện triển khai. Khi tài nguyên dùng chung đạt giới hạn, hệ thống vẫn phải tăng năng lực hoặc thay đổi cách

phân bố tải và dữ liệu. Việc xây dựng cơ chế cô lập, phân quyền, đo lường và quản trị cấu hình cũng làm phát sinh chi phí phát triển và vận hành. Vì vậy, chi phí biên không thể mặc nhiên coi là bằng không, và tổng chi phí không nhất thiết tiến đến một mức cố định khi số thuê bao tăng.

Mối liên hệ kinh tế - kỹ thuật còn thể hiện ở đánh đổi giữa mật độ chia sẻ và mức độ độc lập. Chia sẻ sâu hơn có thể giúp khai thác tài nguyên hiệu quả hơn, nhưng làm tăng yêu cầu kiểm soát ảnh hưởng giữa các thuê bao. Chẳng hạn, dùng chung cơ sở dữ liệu có thể giảm số đơn vị phải quản lý, đồng thời đòi hỏi cơ chế giới hạn truy cập dữ liệu và xử lý cạnh tranh tài nguyên. Ngược lại, tách tài nguyên riêng có thể đáp ứng một số yêu cầu cô lập nhưng làm tăng khối lượng quản trị.

Quan hệ hai đường có điểm giao chỉ đúng khi chi phí khởi đầu và chi phí tăng thêm có sự đánh đổi như đã mô tả. Nếu yêu cầu cô lập, tải hoặc vận hành thay đổi, điểm giao sẽ thay đổi và có thể không xuất hiện trong phạm vi khảo sát. Báo cáo vì vậy dùng đồ thị để giải thích động lực chuyển từ nhiều instance riêng sang Cấp độ 3, không khẳng định một kiến trúc luôn rẻ hơn ở mọi quy mô.

Để lượng hóa TCO sau này, cần ghi rõ thời hạn tính chi phí, cấu hình tài nguyên, số tenant, tải từng tenant và công sức vận hành. Các chi phí dùng chung cần phân bổ theo một quy tắc nhất quán. Chỉ khi có dữ liệu đó mới có thể xác định điểm giao thực tế hoặc tỷ lệ tiết kiệm; phép đo độ trễ hay số request của demo không đủ để suy ra TCO.

1.7 Tổng kết chương

Mô hình bốn cấp độ cho thấy hướng chuyển từ mã và môi trường riêng sang nền tảng chung có cấu hình theo tenant. Đề tài chọn Cấp độ 3: chung mã nguồn, một instance backend, database dùng chung với Tenant ID và metadata được áp dụng lúc chạy. Tenant là tổ chức có quản trị riêng, user là cá nhân trong tổ chức; số user và số instance không tự quyết định tính đa thuê bao. Chương 2 cụ thể hóa các điều kiện này thành kiến trúc Kanban và kế hoạch kiểm chứng.

CHƯƠNG 2 KIẾN TRÚC MỤC TIÊU VÀ KIỂM CHỨNG CẤP ĐỘ 3

Kiến trúc mục tiêu của ứng dụng Kanban được lựa chọn để thể hiện rõ các thuộc tính Cấp độ 3 với số thành phần cần thiết. Phân tích tập trung vào nhận diện tenant, dùng chung ứng dụng và database, tùy biến runtime, cùng việc duy trì cô lập khi nhiều tenant hoạt động. Các cơ chế mở rộng nhiều instance được đặt ở hướng phát triển Cấp độ 4.

2.1 Mục tiêu và phạm vi demo

Mục tiêu của đề tài là thiết kế, xây dựng và demo thành công ứng dụng Kanban đa thuê bao đạt Cấp độ 3. Một instance backend phải phục vụ đồng thời tối thiểu hai tổ chức với các user tương ứng. Các tổ chức có dữ liệu và cấu hình khác nhau, nhưng cùng sử dụng một bộ mã nguồn và một database nghiệp vụ.

Nghiệp vụ tối thiểu gồm tạo workspace, quản lý thành viên, tạo bảng Kanban, cột và thẻ công việc, cập nhật trạng thái thẻ. Workspace tương ứng một tenant. Các thao tác nghiệp vụ cung cấp tình huống để quan sát ranh giới tổ chức; mục tiêu đánh giá chính là kiến trúc phục vụ nhiều tenant và tính đúng đắn khi dùng chung tài nguyên.

Tùy biến runtime cần có ít nhất một thuộc tính giao diện và một quy tắc nghiệp vụ có ý nghĩa, ví dụ nhận diện workspace và giới hạn thẻ đang thực hiện. Những lựa chọn này phải được biểu diễn bằng metadata theo tenant và có thể thay đổi trong khi backend đang chạy. Phạm vi cụ thể phải khớp với chức năng đã hiện thực trong mã nguồn demo.

Bảng 2.1 Yêu cầu kiến trúc của demo

Mã	Yêu cầu	Điều kiện kiểm chứng
R1	Một codebase cho các tenant	Cùng phiên bản backend và cơ chế xử lý; không có bản build riêng cho A hoặc B.
R2	Một instance backend phục vụ nhiều tenant	A và B gửi yêu cầu chồng lấn thời gian vào cùng instance được định danh.
R3	Metadata được áp dụng lúc chạy	Đổi cấu hình A; A nhận hành vi mới, B giữ nguyên; không build hoặc khởi động lại backend.

Mã	Yêu cầu	Điều kiện kiểm chứng
R4	Một database và bảng nghiệp vụ chung	Bản ghi của A, B tồn tại trong cùng bảng với tenant_id khác nhau.
R5	Cô lập dữ liệu và ngữ cảnh	Các phép đọc, ghi và tình huống thiếu hoặc giả tenant bị kiểm soát đúng quyền.

R1–R5 là bộ tiêu chí kiểm chứng của đề tài, được dẫn xuất từ ba nhóm thuộc tính Cấp độ 3. Chúng không thay thế một đánh giá bảo mật toàn diện. Tính sẵn sàng cao, tự động co giãn và tối ưu TCO ở quy mô lớn cần những phép đánh giá riêng và không được suy ra từ việc hoàn thành demo chức năng.

2.2 Lựa chọn cách chia sẻ ứng dụng và dữ liệu

Để đánh giá đúng mục tiêu, cần chỉ rõ cả tầng ứng dụng lẫn tầng dữ liệu. Phương án dùng chung mã nguồn nhưng mỗi tenant có backend và DB riêng tương ứng mốc đối chiếu Cấp độ 2 trong báo cáo. Phương án một backend chung và các bảng có tenant_id được chọn làm kiến trúc mục tiêu Cấp độ 3.

Bảng 2.2 Đối chiếu các phương án lưu trữ phục vụ lựa chọn

Phương án	Đặc điểm	Liên hệ với mục tiêu
Database riêng	Mỗi tenant có database nghiệp vụ độc lập.	Dễ phân tách đơn vị sao lưu; tăng số nguồn dữ liệu phải quản lý. Khi App cũng riêng, thuộc phương án Cấp độ 2 đang đối chiếu.
Schema riêng	Các tenant có nhóm bảng riêng trong cùng database.	Có thêm ranh giới tên; vẫn phải chọn đúng schema và quản lý nhiều nhóm bảng. Không phải phương án demo trọng tâm.
Bảng chung có Tenant ID	Các tenant dùng chung bảng; mỗi hàng nghiệp vụ được gắn tenant_id.	Phù hợp phương án Cấp độ 3 đã chọn; bắt buộc thực thi quyền và giới hạn dữ liệu theo tenant.

Những tên gọi như Silo, Pool, Bridge hoặc Hybrid mô tả cách bố trí tài nguyên ở các tầng. Chúng không tạo ra một thang trưởng thành thay thế và không tương ứng cứng với bốn cấp độ. Trong phạm vi demo, chỉ cần xác định rõ ứng dụng và dữ liệu đều dùng chung; các biến thể dành riêng theo phân khúc khách hàng chưa cần đưa vào kiến trúc triển khai chính.

Lựa chọn bảng chung giúp trình bày trực tiếp bài toán Cấp độ 3: cùng điểm thực thi phải phân biệt hai tenant ở mọi thao tác. Đổi lại, lỗi kiểm soát ngữ cảnh hoặc truy vấn có thể ảnh hưởng nhiều tổ chức. Vì vậy, mức độ dùng chung phải đi kèm kiểm tra quyền, ràng buộc dữ liệu và các tình huống thử chéo tenant; lợi ích chi phí không thay thế cho tính đúng đắn.

2.3 Cấu trúc ứng dụng theo mô hình ba tầng

2.3.1 Tầng người thuê

Tenant A và B có các workspace độc lập. Mỗi workspace có một quản trị viên quản lý thành viên, thông tin tổ chức và các cấu hình được cho phép. User chọn workspace đang thao tác trong phạm vi quyền của mình. Giao diện phải thể hiện rõ workspace hiện hành để tránh nhầm lẫn khi một người có quyền tham gia nhiều tổ chức.

Khi user đổi workspace, ứng dụng phải xác minh lại quyền đối với tenant đích và cập nhật dữ liệu, cấu hình tương ứng. Không được giữ dữ liệu hiển thị hoặc metadata của workspace trước như thể thuộc workspace mới. Các thiết lập cá nhân của user được phân biệt với cấu hình dùng chung của cả tổ chức.

2.3.2 Tầng nhà cung cấp dịch vụ

Các trách nhiệm chính gồm quản lý tenant và thành viên; xác thực và xác lập ngữ cảnh; nghiệp vụ bảng, cột, thẻ; đọc và cập nhật metadata; truy cập dữ liệu có kiểm soát. Những trách nhiệm này có thể được hiện thực trong các mô-đun của cùng backend. Mô hình ba tầng không đòi hỏi chia chúng thành nhiều microservice để chứng minh Cấp độ 3.

Chức năng quản trị nền tảng quản lý tenant và trạng thái sử dụng dịch vụ; chức năng nghiệp vụ xử lý dữ liệu của tenant hiện hành. Sự phân biệt trách nhiệm giúp tránh cấp quyền quản trị toàn nền tảng cho quản trị viên workspace. Trong demo, cả hai có thể cùng nằm trong một ứng dụng nhưng phải có phạm vi quyền rõ ràng.

2.3.3 Tầng instance và hạ tầng

Cấu hình mục tiêu gồm một backend instance phục vụ nghiệp vụ, một Database Server và một database nghiệp vụ dùng chung. Cả tenant A lẫn B kết nối vào cùng backend, không khởi tạo backend theo tenant. Thêm tenant chủ yếu là thêm dữ liệu danh mục tenant, thành viên, metadata và dữ liệu khởi tạo cần thiết.

Ứng dụng và Database Server là hai vai trò kỹ thuật khác nhau. Việc chúng nằm trên cùng máy hoặc các máy khác nhau không thay đổi quan hệ nhiều tenant dùng chung backend và database. Hồ sơ triển khai cần ghi rõ thành phần, phiên bản và cấu hình thực tế để người đọc đối chiếu thiết kế với môi trường demo.

2.4 Xác định và cô lập ngữ cảnh tenant

2.4.1 Phân giải tenant và kiểm tra quyền

Mỗi yêu cầu cần xác định user đang đăng nhập và tenant đang thao tác. Tenant có thể được chọn qua đường dẫn, tên miền, thông tin phiên hoặc claim đã ký; giá trị do client gửi vẫn phải đối chiếu với quyền thành viên của user. Header hoặc tham số tenant_id chỉ biểu diễn tenant được yêu cầu, không tự chứng minh user có quyền truy cập.

Luồng xử lý gồm xác thực danh tính, phân giải tenant, kiểm tra trạng thái tenant và quan hệ thành viên, sau đó thiết lập TenantContext cho phạm vi yêu cầu. Yêu cầu thiếu tenant, tenant không tồn tại hoặc user không có quyền phải bị từ chối trước khi thực hiện nghiệp vụ. Không tự chuyển sang tenant mặc định để tiếp tục xử lý.

TenantContext chỉ chứa những thông tin cần dùng đã được xác minh, chẳng hạn tenant_id và quyền trong workspace. Các lớp nghiệp vụ, truy cập dữ liệu và đọc metadata dùng cùng ngữ cảnh này. Nếu mỗi lớp tự đọc một header hoặc tham số khác nhau, hệ thống có thể áp dụng cấu hình của A trong khi truy vấn dữ liệu của B.

2.4.2 Vòng đời ngữ cảnh trong backend

TenantContext phải được thiết lập trước khi xử lý và giải phóng khi yêu cầu kết thúc, kể cả khi có ngoại lệ. Nếu dùng ThreadLocal trong backend, luồng có thể được tái sử dụng cho yêu cầu khác; ngữ cảnh cũ phải được dọn trong finally. Trong môi trường xử lý bất đồng bộ, cần cơ chế truyền ngữ cảnh phù hợp với mô hình thực thi, không mặc nhiên cho rằng trạng thái của luồng ban đầu tự xuất hiện ở tác vụ mới.

Tác vụ nền hoặc thông điệp, nếu có trong demo, phải mang định danh tenant từ nguồn tin cậy và được xác minh trước khi dùng. Worker thiết lập ngữ cảnh cho từng tác vụ và dọn khi kết thúc. Phần kiểm chứng cần có chuỗi yêu cầu A rồi B

trên tài nguyên xử lý tái sử dụng để phát hiện việc sót ngữ cảnh, thay vì chỉ thử từng tenant riêng lẻ.

Nhật ký phục vụ đối chiếu cần gắn request_id, tenant_id đã xác minh và định danh instance backend. Các trường này giúp nối bằng chứng từ yêu cầu đến thao tác dữ liệu. Log không thay thế kiểm soát quyền và không nên ghi mật khẩu, token đăng nhập hoặc toàn bộ dữ liệu nghiệp vụ nhạy cảm.

2.4.3 Luồng xử lý yêu cầu điển hình

Khi user A tạo thẻ, backend xác thực user, xác nhận user là thành viên của tenant A, thiết lập ngữ cảnh A và đọc metadata của A. Nghiệp vụ kiểm tra quyền tạo thẻ và các quy tắc cấu hình, sau đó ghi thẻ với tenant_id lấy từ ngữ cảnh tin cậy. Sau khi giao dịch kết thúc, backend trả kết quả và giải phóng ngữ cảnh.

Nếu cùng lúc user B tạo thẻ, cùng mã xử lý phải thực hiện các bước tương tự trong ngữ cảnh B. Dữ liệu đi vào cùng bảng nhưng mang tenant_id khác nhau. Giá trị tenant_id do client chèn vào nội dung yêu cầu không được phép ghi đè ngữ cảnh đã xác thực; việc đổi tenant phải đi qua bước kiểm tra quyền tương ứng.

2.5 Thiết kế database dùng chung

2.5.1 Nhóm dữ liệu và khóa tenant

Danh mục tenant lưu định danh và trạng thái tổ chức. Quan hệ thành viên lưu user_id, tenant_id và vai trò trong workspace. Metadata lưu các cấu hình theo tenant. Các thực thể nghiệp vụ như board, column và card thuộc một tenant xác định; chúng cần mang tenant_id trực tiếp hoặc được bảo vệ qua quan hệ sở hữu đã kiểm soát nhất quán. Trong demo, lưu trực tiếp tenant_id trên các bảng nghiệp vụ giúp việc kiểm chứng rõ ràng hơn.

Bảng 2.3 Lược đồ logic tối thiểu được đề xuất

Bảng	Trường chính	Ràng buộc cần thể hiện
tenant	tenant_id, name, status	Định danh duy nhất của tổ chức.
membership	tenant_id, user_id, role	Không trùng cặp tenant và user; kiểm tra quyền theo tổ chức.

Bảng	Trường chính	Ràng buộc cần thể hiện
tenant_config	tenant_id, config_key, config_value, version	Metadata được chọn và cập nhật theo đúng tenant.
board	tenant_id, board_id, name	Board thuộc một tenant; định danh phục vụ liên kết trong tenant.
kanban_column	tenant_id, column_id, board_id	Board được tham chiếu phải cùng tenant.
card	tenant_id, card_id, column_id, title	Column được tham chiếu phải cùng tenant.

Lược đồ trên là mô hình logic mục tiêu, không mặc nhiên là tên bảng đã tồn tại trong mã nguồn. Khi hiện thực, cần thống nhất khóa, kiểu dữ liệu và tên đối tượng giữa migration, mã truy cập và tài liệu. Chứng cứ database phải cho thấy bản ghi của ít nhất hai tenant trong cùng bảng nghiệp vụ.

2.5.2 Toàn vẹn liên kết và truy vấn theo tenant

Mọi thao tác đọc, cập nhật hoặc xóa đối tượng phải bị giới hạn theo tenant hiện hành và quyền nghiệp vụ. Với liên kết cha–con, cần ngăn trường hợp card của A trỏ tới column của B. Có thể dùng khóa duy nhất theo cặp (tenant_id, object_id) và khóa ngoại mang cùng tenant_id, kết hợp kiểm tra nghiệp vụ. Chỉ kiểm tra object_id tồn tại chưa bảo đảm đối tượng thuộc đúng tổ chức.

Các ràng buộc duy nhất theo nghiệp vụ cần xét phạm vi tenant, ví dụ cùng một tên board có thể hợp lệ ở hai tổ chức khác nhau. Chỉ mục bắt đầu bằng tenant_id có thể phù hợp với truy vấn thường xuyên theo tenant và khóa đối tượng; lựa chọn cuối cùng cần dựa vào truy vấn thực tế và kế hoạch thực thi. Có tenant_id trong chỉ mục giúp truy cập hiệu quả nhưng không phải cơ chế cấp quyền.

Cần kiểm tra cả danh sách, tìm kiếm, chi tiết, thống kê và các thao tác ghi. Một API danh sách đúng không chứng minh API xem chi tiết bằng ID hoặc cập nhật hàng loạt cũng đúng. Các đường truy cập dữ liệu gián tiếp như tải tệp, xuất dữ liệu và cache, nếu được triển khai, cũng phải dùng cùng phạm vi tenant.

2.5.3 Cơ chế kiểm soát tại database

Với PostgreSQL, Row-Level Security (RLS) có thể bổ sung lớp kiểm soát hàng theo ngữ cảnh tenant. Thiết kế cần xét cả hàng được nhìn thấy hoặc sửa và hàng

mới được ghi. RLS không tự xác thực tenant; ứng dụng vẫn chịu trách nhiệm thiết lập ngữ cảnh đúng. Tài khoản chạy ứng dụng và chính sách truy cập phải được kiểm tra trong môi trường demo, thay vì chỉ kiểm thử bằng tài khoản quản trị.

Chủ bảng, superuser hoặc vai trò có BYPASSRLS có thể không chịu chính sách như tài khoản thông thường. Do đó, cần xác nhận quyền thực tế và cấu hình áp dụng RLS, bao gồm FORCE ROW LEVEL SECURITY khi phù hợp. Tài liệu PostgreSQL là căn cứ đối chiếu cơ chế này; việc sử dụng RLS trong ứng dụng chỉ được ghi nhận là đã hiện thực khi có migration và kiểm thử bằng vai trò thực tế [4].

Nếu ngữ cảnh được truyền vào database bằng SET LOCAL, việc thiết lập và truy vấn nghiệp vụ phải nằm trong cùng transaction. Giá trị cục bộ kết thúc cùng transaction; trạng thái session trước đó có thể có hiệu lực trở lại. Vì vậy, việc quản lý connection pool vẫn cần bảo đảm không tồn tại ngữ cảnh session sai, đồng thời chính sách phải xử lý rõ tình huống thiếu hoặc rỗng tenant [5].

RLS là một lựa chọn triển khai cơ chế cô lập của phương án bảng chung; nó không phải tên một cấp độ trưởng thành. Bộ tiêu chí Cấp độ 3 cần kiểm chứng dữ liệu được phân tách đúng, bất kể cơ chế cụ thể được dùng là gì. Khi chọn RLS, phải kiểm tra hiệu lực của cả ứng dụng, vai trò database và chính sách, không chỉ sự tồn tại của câu lệnh khai báo.

2.6 Tùy biến runtime qua metadata

2.6.1 Những điểm tùy biến được hỗ trợ

Metadata là dữ liệu mô tả cách ứng dụng hoạt động hoặc hiển thị cho tenant. Các trường như tên workspace, màu nhận diện, tính năng được bật và giới hạn nghiệp vụ được lưu cùng tenant_id. Mã nguồn định nghĩa trước cách hiểu những trường này; metadata chỉ chọn biến thể đã hỗ trợ, không thay thế bằng một bộ mã tùy biến riêng cho mỗi tổ chức.

Ví dụ, metadata giao diện xác định tên và màu workspace; metadata nghiệp vụ xác định có áp dụng giới hạn thẻ đang thực hiện hay không và giới hạn là bao nhiêu. A và B có giá trị khác nhau trong cùng cấu trúc dữ liệu. Khi yêu cầu đi vào backend, tenant hiện hành quyết định bộ metadata được chọn và quy tắc được áp dụng.

Cần phân biệt metadata với dữ liệu nghiệp vụ thông thường. Việc tạo một board hoặc sửa tiêu đề card thể hiện thay đổi dữ liệu công việc; riêng thao tác đó chưa chứng minh tùy biến hành vi ứng dụng. Phép thử runtime phải làm rõ một cấu hình đã thay đổi giao diện hoặc cách xử lý nghiệp vụ được nền tảng hỗ trợ.

2.6.2 Cập nhật cấu hình khi ứng dụng đang chạy

Luồng cập nhật gồm kiểm tra quyền quản trị trong tenant, kiểm tra tính hợp lệ của giá trị, lưu metadata và làm cho lần xử lý tiếp theo đọc được phiên bản phù hợp. Không cần sửa mã nguồn, build lại hoặc tạo instance riêng cho tenant. Nếu cấu hình được cache, cần quy tắc hết hạn hoặc invalidation có thể quan sát trong thử nghiệm.

Cấu hình phải có phạm vi và ràng buộc rõ ràng. Ví dụ, giới hạn thẻ không nhận số âm, giá trị màu phải hợp lệ và khóa cấu hình không được tùy tiện biến thành lệnh thực thi. Khi cập nhật sai, hệ thống trả lỗi có kiểm soát và giữ cấu hình hợp lệ trước đó; không áp dụng một phần làm tenant rơi vào trạng thái khó xác định.

Quy tắc nghiệp vụ được áp dụng ở backend, còn giao diện phản ánh trạng thái đó cho user. Nếu một tính năng bị tắt, gọi trực tiếp API cũng phải tuân theo quyền và cấu hình. Giấu nút hoặc menu chỉ thay đổi cách hiển thị, không tạo ra ranh giới chức năng giữa các tenant.

2.6.3 Phân biệt với cấu hình của Cấp độ 2

Ở Cấp độ 2 theo phương án đối chiếu, App A nạp cấu hình A trước khi chạy và App B nạp cấu hình B. Ở Cấp độ 3, cùng App đang chạy xử lý liên tiếp hoặc đồng thời yêu cầu A và B, mỗi lần chọn metadata theo tenant. Do đó, vị trí lưu cấu hình chưa đủ phân biệt: cần quan sát cùng instance có thay đổi hành vi theo ngữ cảnh trong lúc phục vụ hay không.

Phép demo rõ nhất là giữ nguyên định danh instance và phiên bản backend, mở A và B trong hai phiên độc lập, thay cấu hình A rồi thực hiện lại cùng loại thao tác. Nếu A nhận hành vi mới, B giữ nguyên, không có lần build hoặc restart xen giữa, bằng chứng đó liên kết được metadata, runtime và sự độc lập giữa các tenant.

2.7 Tài nguyên dùng chung và giới hạn của kiến trúc

Một instance và database dùng chung làm các tenant cạnh tranh CPU, bộ nhớ, kết nối và I/O. Tenant có tải lớn có thể làm tăng thời gian chờ của tenant khác,

thường gọi là hiệu ứng hàng xóm ồn ào. Vì vậy, cô lập dữ liệu đúng không đồng nghĩa đã cô lập hiệu năng hoặc bảo đảm tính sẵn sàng cao.

Trong phạm vi demo, có thể áp dụng giới hạn lưu lượng hoặc mức đồng thời theo tenant và theo dõi độ trễ, tỷ lệ lỗi riêng cho A và B. Việc đánh giá tập trung vào các cơ chế tài nguyên được hiện thực trong ứng dụng và tác động của chúng đến từng tenant.

Nếu có cache, khóa dữ liệu hoặc cấu hình phải chứa tenant_id cùng định danh đối tượng phù hợp. Không dùng chung cache entry chỉ theo card_id khi có khả năng nhầm phạm vi tổ chức. Xóa hoặc làm mới metadata của A phải giữ cấu hình B đúng; việc thử tùy biến runtime cần bao gồm trường hợp đọc sau cập nhật.

Sao lưu database chung không tự cung cấp khả năng khôi phục riêng một tenant. Cập nhật lược đồ cũng có thể ảnh hưởng tất cả tenant. Báo cáo cần nêu đây là hạn chế vận hành của phương án dùng chung và xác định chiến lược tương thích, sao lưu, phục hồi ở mức phù hợp; không dùng các tên gọi như zero downtime như một kết quả khi chưa được đánh giá.

2.8 Kế hoạch kiểm chứng demo theo tiêu chí Cấp độ 3

2.8.1 Thiết lập và bằng chứng cần thu thập

Môi trường kiểm chứng cần ghi phiên bản hoặc commit của mã nguồn, cách khởi chạy backend, định danh instance, cấu hình database và tài khoản ứng dụng. Tạo tenant A, B, mỗi tenant có quản trị viên và thành viên; chuẩn bị dữ liệu có thể phân biệt và cấu hình khác nhau. Tài khoản dùng cho thử chéo tenant chỉ có quyền tại tenant của mình.

Mỗi ca kiểm chứng lưu đầu vào, điều kiện trước khi chạy, thời điểm, kết quả kỳ vọng và kết quả quan sát. Bằng chứng phù hợp gồm yêu cầu và phản hồi API, log có tenant_id và instance_id, ảnh giao diện, truy vấn đối chiếu dữ liệu và phần mã hoặc migration thực thi cơ chế. Ảnh giao diện chỉ là một phần bằng chứng, không đủ xác nhận backend hoặc database được dùng chung.

Bảng 2.4 Ma trận tiêu chí và bằng chứng cần có

Tiêu chí	Cách kiểm chứng	Bằng chứng
Mã nguồn chung và metadata runtime	Đổi cấu hình A trên hệ thống đang chạy; lặp thao tác của A và B.	Phiên bản code; metadata trước/sau; hành vi A thay đổi, B giữ nguyên; định danh instance không đổi.
Một instance phục vụ đồng thời nhiều tenant	Gửi yêu cầu A, B chồng lấn thời gian vào cùng backend.	Cấu hình triển khai; log request có tenant_id, timestamp và cùng instance_id.
Database và bảng chung có Tenant ID	Tạo dữ liệu A, B rồi đối chiếu cùng bảng.	Lược đồ; migration; bản ghi có tenant_id khác nhau trong cùng database.
Cô lập dữ liệu và ngữ cảnh	Thử đọc/ghi chéo tenant, thiếu hoặc giả tenant; xen kẽ A, B.	Yêu cầu/phản hồi; log; kiểm tra dữ liệu trước/sau; chính sách và vai trò database nếu dùng RLS.

2.8.2 Các ca kiểm chứng chức năng cốt lõi

Bảng 2.5 Các ca kiểm chứng tối thiểu

Mã	Thao tác	Kết quả kỳ vọng
T01	A và B tạo, đọc thẻ trong workspace của mình.	Mỗi bên thao tác đúng quyền và chỉ thấy dữ liệu thuộc phạm vi của mình.
T02	Cập nhật metadata giao diện của A khi backend đang chạy.	A hiển thị cấu hình mới theo cơ chế refresh; B giữ nguyên; không build hoặc restart.
T03	Đổi quy tắc nghiệp vụ của A rồi gọi API ở A và B.	Backend áp dụng quy tắc mới cho A, không áp dụng nhầm cho B.
T04	Gửi request A, B chồng lấn thời gian.	Cùng instance xử lý hai tenant; dữ liệu và cấu hình không lẫn.
T05	User A dùng ID đối tượng của B để đọc, sửa hoặc xóa.	Từ chối hoặc không trả đối tượng theo quy ước API; dữ liệu B không đổi.
T06	Thiếu tenant hoặc sửa tenant_id ở header, URL hay body.	Không vượt quyền; không tự dùng tenant mặc định; không ghi vào tenant giả.

Mã	Thao tác	Kết quả kỳ vọng
T07	Xử lý A rồi B trên worker hoặc connection tái sử dụng.	Không còn ngữ cảnh A khi phục vụ B; kiểm tra cả nhánh lỗi và rollback.
T08	Tạo liên kết cha–con chéo tenant.	Không tạo được card A tham chiếu column B; dữ liệu giữ toàn vẹn.
T09	Gọi API cập nhật cấu hình bằng user không có quyền quản trị.	Bị từ chối; cấu hình của cả hai tenant giữ nguyên.

Các kết quả trong Bảng 2.5 là điều kiện kỳ vọng trước khi chạy thử. Khi ghi kết quả thực tế, cần đối chiếu đầy đủ cả phản hồi API lẫn trạng thái dữ liệu; một lỗi HTTP không tự chứng minh thao tác chưa gây tác động. Nếu có cơ chế cache hoặc tác vụ nền, bổ sung ca kiểm chứng tương ứng với thành phần thực sự tồn tại.

2.8.3 Trình tự demo và đánh giá kết quả

Trình tự trình bày bắt đầu bằng cấu hình triển khai một backend và một database. Sau đó đăng nhập A và B ở hai phiên độc lập, thể hiện cùng chức năng Kanban với dữ liệu và cấu hình riêng. Tiếp theo thay đổi metadata A khi backend vẫn chạy, quan sát thay đổi ở A và kiểm tra B giữ nguyên. Cuối cùng thực hiện yêu cầu đồng thời và các thao tác đọc, ghi chéo tenant để đối chiếu cơ chế cô lập.

Việc đánh giá cần trả lời từng yêu cầu R1–R5 bằng bằng chứng cụ thể. R1 và R3 chứng minh mã nguồn chung cùng tùy biến runtime; R2 chứng minh chia sẻ instance; R4 và R5 chứng minh database chung có phân tách logic và ngữ cảnh được quản lý đúng. Các ca chưa chạy hoặc không đủ bằng chứng được ghi là chưa xác nhận, không được thay bằng kết quả suy ra từ sơ đồ thiết kế.

Khi đo hiệu năng hỗ trợ, ghi rõ số tenant, số user đồng thời, tải mỗi tenant, kích thước dữ liệu, thời gian chạy và cấu hình tài nguyên. Báo cáo độ trễ và tỷ lệ lỗi theo tenant bên cạnh số tổng. Phép đo này giúp nhận diện giới hạn của ứng dụng chung; tự nó không chứng minh mở rộng ngang hoặc hiệu quả TCO ở quy mô thương mại.

Phần thực nghiệm hoàn chỉnh cần trình bày thiết lập đã dùng, kết quả quan sát của từng ca, bằng chứng và nhận xét. Kết luận đạt Cấp độ 3 chỉ được đưa ra sau khi ba nhóm thuộc tính đều có chứng cứ tương ứng. Bộ tiêu chí ở đây tạo căn cứ

trực tiếp cho việc hiện thực và đánh giá ứng dụng, không thay cho kết quả chạy thử.

2.9 Hướng phát triển lên Cấp độ 4

Sau khi kiểm chứng Cấp độ 3, bước phát triển tiếp theo là nhân bản backend sau bộ cân bằng tải. Các instance dùng cùng phiên bản ứng dụng và cơ chế xác lập tenant; trạng thái phiên và cấu hình không phụ thuộc sai vào bộ nhớ cục bộ của một instance. Nếu triển khai bằng container/pod, phải bảo đảm thêm pod mới vẫn áp dụng đúng tenant và metadata.

Tầng dữ liệu cần được đánh giá riêng theo tải. Có thể mở rộng tài nguyên, tối ưu truy vấn, thêm bản sao đọc hoặc phân vùng dữ liệu; database phân tán là một phương án khi có yêu cầu và mô hình nhất quán phù hợp. Không tự giả định rằng tăng số backend sẽ giải quyết giới hạn của một database chung.

Để kết luận về Cấp độ 4, cần kiểm chứng lưu lượng được phân phối qua nhiều instance, khả năng tăng năng lực khi bổ sung tài nguyên, hành vi khi instance gặp lỗi và tính đúng đắn của ngữ cảnh trên toàn bộ luồng. Khả năng tự động co giãn, nếu xây dựng, phải được đánh giá thêm qua chính sách kích hoạt và kết quả quan sát. Những nội dung này thuộc hướng mở rộng sau mục tiêu demo Cấp độ 3.

2.10 Tổng kết chương

Kiến trúc mục tiêu sử dụng một backend và database chung cho nhiều tenant, đồng thời duy trì phạm vi dữ liệu và cấu hình theo tổ chức. TenantContext kết nối xác thực với nghiệp vụ, metadata quyết định các biến thể lúc chạy, và Tenant ID hỗ trợ phân tách dữ liệu trên bảng chung. Ma trận kiểm chứng liên kết trực tiếp những cơ chế này với bằng chứng cần có để đánh giá Cấp độ 3. Cân bằng tải, mở rộng ngang và phân bố dữ liệu được xác định là hướng phát triển Cấp độ 4.

Tài liệu tham khảo

[1] Mell, P. và Grance, T. (2011). The NIST Definition of Cloud Computing. NIST SP 800-145. https://doi.org/10.6028/NIST.SP.800-145

[2] Chong, F. và Carraro, G. (2006). Architecture Strategies for Catching the Long Tail. Microsoft, April 2006. Mô hình trưởng thành SaaS bốn cấp độ. https://msdn.microsoft.com/en-us/library/aa479069.aspx

[3] Kabbedijk, J., Bezemer, C.-P., Jansen, S. và Zaidman, A. (2015). Defining Multi-Tenancy: A Systematic Mapping Study on the Academic and the Industrial Perspective. Journal of Systems and Software, 100, 93–103. https://doi.org/10.1016/j.jss.2014.10.034

[4] PostgreSQL Global Development Group. PostgreSQL 17 Documentation, Row Security Policies. https://www.postgresql.org/docs/17/ddl-rowsecurity.html. Truy cập ngày 07/11/2026.

[5] PostgreSQL Global Development Group. PostgreSQL 17 Documentation, SET. https://www.postgresql.org/docs/17/sql-set.html. Truy cập ngày 07/11/2026.